

AI Revenue Recovery & Payment Intelligence Platform

Scoped Build Blueprint — Razorpay AI Buildathon 2026

This turns the uploaded vision document into something you can actually finish, run end-to-end, and defend in a panel interview by Sept 5 — while keeping the parts of the original vision that make it genuinely impressive.

Part A — The Scoping Decision

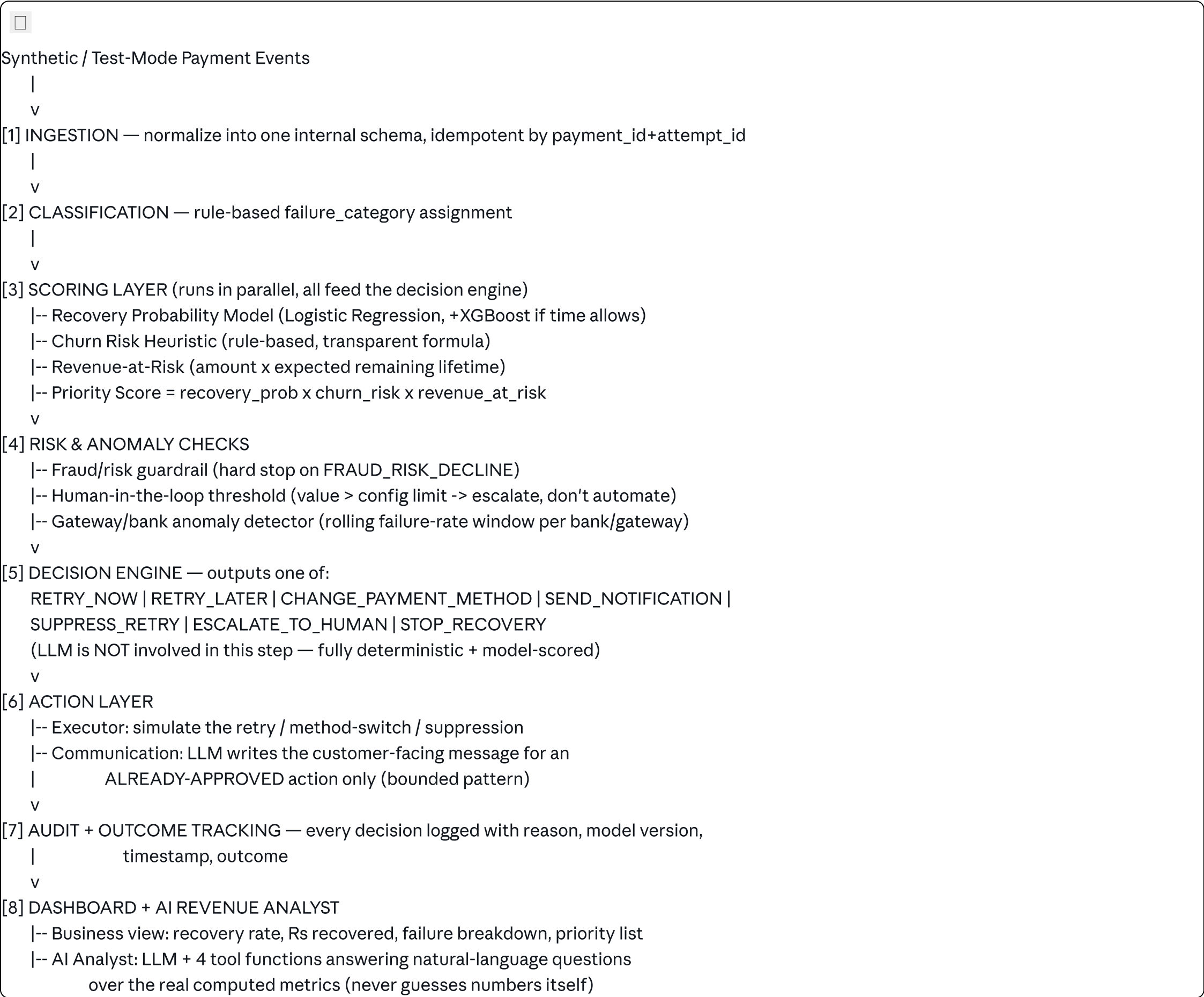
The original document is a strong **north-star roadmap**, not a hackathon scope. Its own Section 42 already stages this into 8 phases — we’re building a well-chosen slice of Phase 1, 2, 4, and 5, and explicitly documenting the rest as roadmap. This is not a downgrade of the idea — it’s the difference between a system that runs and one that doesn’t.

#	Feature (from original doc)	Status	Reasoning
1	Payment failure ingestion	BUILD	Core — nothing works without it
2	Failure classification (rules)	BUILD	Cheap, high value, judges expect this
3	Recovery probability model	BUILD (1 model)	Real ML story — Logistic Regression baseline, XGBoost if time allows. NOT all 5 candidate model types.
4	Customer behavior profile	BUILD (lightweight)	Feed as features into #3, not a separate system
5	Intelligent retry timing (ML)	ROADMAP	Needs rich longitudinal per-customer history a one-shot synthetic batch can't credibly provide — use category-based fixed windows instead
6	Next-best payment method	BUILD (heuristic, not ML)	Cheap lookup against synthetic history — high narrative value for low cost
7	Dynamic customer communication	BUILD	LLM generates text only, only after decision engine approves — this is the bounded-architecture proof point
8	Churn prediction (full ML model)	BUILD (heuristic, not trained model)	A trained churn model needs more longitudinal data than a synthetic batch credibly has — a transparent heuristic (recent failures + subscription age) is more honest than a fake-precision model
9	Revenue at risk	BUILD	Simple arithmetic, huge narrative payoff
10	Priority score	BUILD	Combines #3/#8/#9 — cheap, high value
11	Bank/gateway anomaly detection	BUILD (rule-based)	A rolling-window threshold check, NOT real anomaly-detection ML — still a great demo moment
12	Retry suppression on outage	BUILD	Pairs directly with #11
13	Fraud/risk guardrails	BUILD	This IS the "bounded and gated" bar — non-negotiable
14	Central decision engine	BUILD	The spine of the system
15	Human-in-the-loop threshold	BUILD	Cheap (one config value), huge credibility signal
16	Explainable AI	BUILD (coefficients, not SHAP)	Logistic regression coefficients ARE explainable — SHAP is a nice-to-have polish item only if time remains
17	Outcome tracking	BUILD	Needed for every metric you'll report
18	ML feedback loop (retraining)	ROADMAP	Needs real production traffic over time — document the design, don't build a live retrain loop
19	Model monitoring (drift, alerts)	ROADMAP	Meaningless on a static synthetic batch — mention in architecture doc as "designed for"
20	Data drift detection	ROADMAP	Same as above
21	Root cause analysis	BUILD (folded into #22)	It's just another question the AI Analyst answers
22	AI Revenue Analyst (tool-calling LLM)	BUILD (4 tools)	Cheap on top of infra that already exists, exceptional demo moment, textbook-correct "AI Judgment"
23	What-if simulation	ROADMAP	Real feature, real time cost — cut for now
24	A/B testing	ROADMAP	Needs live traffic variance you won't have
25	Reinforcement learning	ROADMAP	The doc itself says: only after a strong supervised baseline. Correct — skip entirely.

#	Feature (from original doc)	Status	Reasoning
26	Segment-level strategy learning	ROADMAP	Depends on #18/#24 existing first
27–29	Dashboards (business + customer)	BUILD (simplified, one page)	Essential for the demo
30	Idempotency	BUILD	Cheap, and it's exactly the kind of detail a technical panel probes
31	Kafka/event-bus architecture	ROADMAP	A synthetic one-shot batch does not need a message bus — document as the production evolution path
32	Audit logs	BUILD	Directly required by the track’s bar

Net result: ~20 of the 32 features, all genuinely working, end-to-end, on real (if synthetic) data — instead of 32 features where most are stubs. This is a stronger submission, not a smaller one.

Part B — System Architecture (scoped)



The one rule that matters most: nothing in layers 1–5 involves an LLM. Scoring and decisions are deterministic/statistical and fully auditable. The LLM only appears in layer 6 (writing already-approved messages) and layer 8 (answering questions using tool calls over real data). This is the single detail most likely to earn you credit on “AI Judgment” — you can point at the code and show the LLM literally cannot execute a financial action.

Part C — Data Model

Reuse the schema from the original document almost as-is — it’s well designed:

payment_id, customer_id, subscription_id, amount, currency, payment_method, gateway, bank, failure_code, failure_category, timestamp, hour, day_of_week, previous_success_count, previous_failure_count, customer_age_days, subscription_value, retry_count, retry_delay, recovered, recovery_time, churned

Failure taxonomy (consolidated to 9 categories — enough breadth, not so many that per-category data gets too thin to say anything meaningful):

INSUFFICIENT_FUNDS, EXPIRED_CARD, INVALID_CARD, AUTHENTICATION_FAILURE, BANK_DECLINED, GATEWAY_OR_NETWORK_FAILURE, FRAUD_RISK_DECLINE, PAYMENT_METHOD_FAILURE, TEMPORARY_BANK_ISSUE

Synthetic generator requirements:

Generate 150–250 events across the taxonomy above, in a realistic (not uniform) distribution

Give each customer a short synthetic history (5–20 prior payments) so previous_success_count / previous_failure_count / recovery_rate are meaningful, not placeholder zeros

Inject one deliberate "bank outage" scenario: a specific bank/gateway's failure rate should spike from ~2% to ~35–40% over a short synthetic time window, then recover. This is what your anomaly-detection feature demonstrates live — use the original document's own example numbers (2% -> 3% -> 12% -> 38%) as the injected curve.

Part D — The ML Layer (kept honest and small)

Recovery Probability Model

Target: recovered (0/1)

Features: one-hot failure_category, previous_success_count, previous_failure_count, customer_age_days, subscription_value, retry_count, hour, day_of_week, payment_method

Model: Logistic Regression as the shipped baseline. If time allows, train XGBoost too and report the lift — this mirrors exactly the "honest, held-out metrics" bar the sibling Risk Manager track states explicitly, and it's good practice regardless of track.

Evaluation: train/test split, report **precision, recall, F1, ROC-AUC** on the held-out set — never just accuracy. State the numbers plainly, including where the model is weak.

Churn Risk (heuristic, not a trained model — say this openly in your docs)

churn_risk = normalize(recent_failure_count) x normalize(1 / (subscription_age_days + 1))

Being transparent that this is a documented heuristic, not a model dressed up to look like one, is a credibility point, not a weakness — a fabricated-precision churn model on thin synthetic data is worse than an honest formula.

Revenue at Risk

revenue_at_risk = subscription_value x assumed_remaining_months

State your assumption for assumed_remaining_months explicitly (e.g., 6) rather than hiding it.

Priority Score

priority = recovery_probability x churn_risk x revenue_at_risk
--

, then rank descending.

Part E — Guardrails (this section is what wins the track’s stated bar)

Rule	Implementation
Never auto-retry a fraud/risk decline	Hard stop in code, before any model or LLM is consulted — not a "usually" rule
Human-in-the-loop above a value threshold	Configurable constant (e.g. Rs 50,000+) routes to ESCALATE_TO_HUMAN regardless of recovery probability
Hard retry cap	Max 3–4 attempts per subscription, then STOP_RECOVERY and escalate
Anomaly-triggered suppression	If a bank/gateway's rolling failure rate crosses ~3x its baseline, SUPPRESS_RETRY for all events on that bank/gateway until it normalizes
Contact-frequency cap	Never message the same customer more than once inside a defined window
Idempotency	payment_id + attempt_id as a unique key — duplicate webhook deliveries must produce exactly one decision, not one per delivery

Rule	Implementation
Every decision logged with a reason	One plain-English sentence per decision, stored with model version + timestamp — this is your audit trail

Part F — The LLM Layer (2 uses only, both bounded)

- 1. Customer Communication** — takes an *already-approved* action + failure category, generates the message text only. It cannot choose the action, cannot change the amount, cannot decide to retry. Feed it a strict template + the approved action as context.
- 2. AI Revenue Analyst** — a small tool-calling layer with exactly these functions to start:

- `get_recovery_metrics()` — overall recovery rate, Rs recovered, Rs at risk
- `get_failure_breakdown()` — counts/rates per failure_category
- `get_gateway_health()` — current anomaly status per bank/gateway
- `get_top_priority_cases(n)` — the current top-N by priority score

The LLM calls these, then writes a natural-language answer to questions like “*why did recovery drop*” or “*which bank is causing the most failures right now*” — it never invents a number that didn't come from a tool call. This is the single most impressive-per-hour-invested feature in the whole build — it's cheap once the metrics functions exist, and it's exactly the kind of LLM use that reads as genuine judgment rather than a bolted-on chatbot.

Part G — Dashboard (one page, two views)

- Business view:** total failed payments, Rs at risk, Rs recovered, recovery rate, failure breakdown by category, current gateway/bank health, an “AI Insights” callout line (e.g., “*Bank X failure rate up 4x — retries suppressed, ~Rs X protected*”), and the top-priority case list.
- Case drill-down:** for any single payment — failure reason, recovery probability, churn risk, revenue at risk, the decision taken, and a “why this decision” explanation pulled straight from the audit log. This is the explainability payoff and it costs almost nothing extra to expose once the audit log exists.

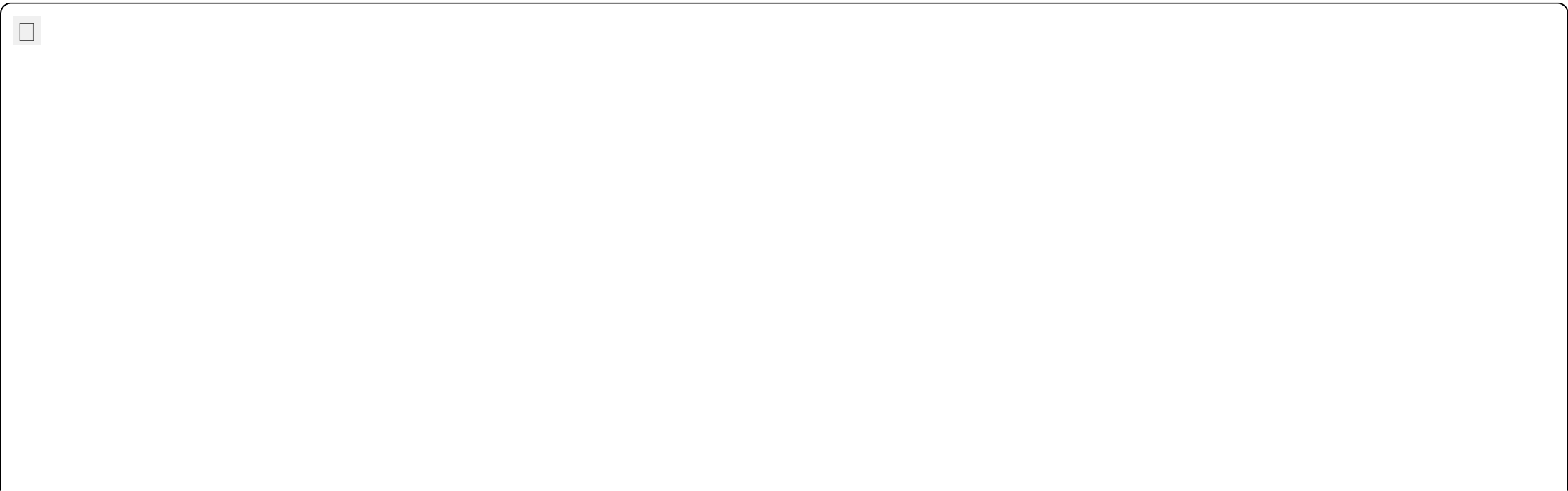
Part H — Tech Stack

Ship with (scoped for a 10-day build):

- Backend: Python + FastAPI + Pydantic
- ML: scikit-learn (Logistic Regression), optionally `xgboost`
- DB: SQLite (genuinely sufficient at this scale — resist the urge to add more)
- Scheduler: a simple in-process job runner (APScheduler) — no task queue needed
- LLM: any provider, abstracted behind one function so it's swappable
- Dashboard: Streamlit (fastest path to something that looks good) or a small React page
- Tests: pytest, covering the guardrails specifically

Explicitly deferred — document these as the production evolution path, do not build them: Kafka/Redpanda, Redis, Celery, Docker/CI-CD pipelines, Prometheus/Grafana, multi-cloud deployment, a model registry, drift detection, A/B testing infrastructure, reinforcement learning. Naming these in your architecture doc as “designed for, deliberately out of scope for the hackathon window” is a feature, not an omission — it shows you know the difference between a demo and a production system, which is precisely what experienced reviewers are checking for.

Part I — Repo Structure



ai-revenue-recovery/
README.md
docs/
 architecture.md
 what-broke.md
 roadmap.md <- explicitly lists the deferred features from Part A and why
data/
 generate_synthetic.py
src/
 ingestion/
 classification/
 scoring/ <- recovery model, churn heuristic, revenue-at-risk, priority
 guardrails/
 decision_engine/
 communication/ <- bounded LLM message generation
 analyst/ <- AI Revenue Analyst tool-calling layer
 audit/
dashboard/
tests/
requirements.txt
.env.example

Part J — Evaluation: Prove It Actually Worked

Borrow this framing directly from the original document’s Section 38 — it’s the strongest part of the whole plan and costs nothing extra to produce:

Baseline vs AI comparison. Run the same synthetic batch through (a) a naive fixed-interval retry baseline and (b) your full system. Report both recovery rate and Rs recovered for each, and state the improvement plainly (e.g., *“Baseline: 61% recovery, Rs 10L. AI system: 76% recovery, Rs 14L. Incremental: Rs 4L.”*). A judge can evaluate this claim in ten seconds — it’s worth more than any architecture diagram.

The exception list. Report every case your system could not resolve, with a reason. This is explicitly what the track rewards over a cherry-picked success story.

Part K — Build Plan (Today -> Sept 5)

Day	Focus
1	Data model, synthetic generator (including the injected bank-outage scenario), classification rules
2	Recovery probability model (train + evaluate), revenue-at-risk, priority score
3	Decision engine + guardrails (risk hard-stop, human-in-the-loop threshold, retry cap) + audit log
4	Anomaly detection + retry suppression
5	LLM layer: bounded communication + AI Revenue Analyst tool-calling
6	Dashboard: business view + case drill-down
7	Full end-to-end run on the complete batch; capture the baseline-vs-AI numbers
8	Tests (guardrails, idempotency) + fix what breaks
9	README, architecture.md, what-broke.md, roadmap.md, record the pitch video
10	Buffer, rehearse, submit early

Part L — What to Say in the Pitch/Interview About the Cut Features

Don’t hide that you scoped down — say it out loud, it’s a strength:

“The full vision includes churn ML, retry-timing ML, A/B testing, and production infra like Kafka and a model registry. For a 10-day build we deliberately shipped the core loop — ingestion, classification, a real trained recovery model, guardrails, anomaly detection, and two bounded LLM features — end-

to-end and fully tested, and documented exactly how it extends to the rest. We'd rather show you something real than something wide and half-working."

That sentence, delivered with a working demo behind it, is a better signal than a slide listing 32 features.